

UDC 004.451.2

Shago P. E.

EEVDF Linux Process Scheduler implementation via sched_ext

Recommended for publication by associate professor Korkhov V. V.

1. Introduction. The research of scheduling algorithms remains a vital direction in systems programming, since it is the scheduler that governs resource utilization, system responsiveness, and the predictability of the operating system. Advances in scheduler design directly improve system performance, as evidenced by the Linux kernel's recent adoption of EEVDF over CFS, yielding measurable latency and throughput gains across billions of deployed systems [1].

Furthermore, the emergence of sched_ext represents a significant frontier in Linux scheduler research. sched_ext [2] is a framework, merged into the Linux kernel mainline as of version 6.12, that exposes a set of hooks within the scheduling subsystem to which user-defined scheduling policies can be attached at runtime via eBPF programs.

Rather than requiring kernel modification and recompilation, sched_ext allows custom scheduling logic to be loaded dynamically on a live system, decoupling policy from the kernel's core mechanism. This enables rapid prototyping of novel scheduling strategies. A notable predecessor to this approach is ghOSt [3], a framework developed at Google that delegates scheduling decisions to userspace agents.

Conducted research combines reimplementing of the Earliest Eligible Virtual Deadline First (EEVDF) algorithm, the Linux fair-class scheduler since version 6.6 [1], through the sched_ext framework.

2. Model. EEVDF, formalized by Stoica and Abdel-Wahab in [4, 5], is a proportional-share discipline for a time-shared resource dispatched in quanta of size at most q . Each active client i carries a positive weight w_i , and the aggregate active weight at time t is

$$W(t) = \sum_{j \in A(t)} w_j, \quad (1)$$

Shago Pavel Evgenovich – student, Saint-Petersburg state University; e-mail: pavel@shago.dev, phone: +7(901)7532145

where $A(t)$ denotes the set of clients competing for service. The ideal reference is a fluid system in which client i continuously receives instantaneous share $w_i/W(t)$. EEVDF captures this entitlement through system virtual time

$$V(t) = \int_{t_0}^t \frac{1}{W(\tau)} d\tau, \quad (2)$$

so that one unit of V corresponds to one unit of service per unit weight.

Let t_0^i denote the instant at which client i most recently became active and $s_i(t_0^i, t)$ the actual service it has accumulated since that instant. When client i submits a request of size r at time t , the algorithm assigns a virtual eligible time

$$v_e = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i} \quad (3)$$

and a virtual deadline

$$v_d = v_e + \frac{r}{w_i}. \quad (4)$$

The request is considered runnable only once $v_e \leq V(t)$. Among all eligible requests, EEVDF dispatches the one with the smallest v_d . This rule couples fairness with latency. Clients that have received less than their fluid-model share become eligible sooner, while larger weights compress the term r/w_i , producing earlier deadlines for the same request size. The lag of client i , defined as the signed deviation between ideal and actual service, satisfies

$$|\text{lag}_i(t)| < q \quad (5)$$

in steady state, an optimal bound for any discrete proportional-share scheduler operating with quantum q [4, 5].

3. Mapping. The proposed `sched_ext` variant implements EEVDF as a discrete, event-driven scheduler whose state consists of a global virtual-time reference and per-task virtual service timestamps.

In the implementation, the client is a runnable task, w_i is the task's scheduler weight (kernel nice value), and its virtual position is stored in thread-local `scx.dsqr_vtime`. Global state is minimal and consists only of `vtime_now`, which plays the role of the system virtual-time frontier, and `total_weight`, which tracks the total runnable weight.

The theoretical request size r is represented by the execution slice Δ assigned to a task by `sched_ext` subsystem. On enqueue, the scheduler computes a discrete analogue of the EEVDF eligibility condition,

$$VE_i = \max(v_i, V_{\text{now}} - \Delta), \quad (6)$$

where v_i is the task’s stored virtual time and $V_{\text{now}} \equiv \text{vtime_now}$. This prevents a task from re-entering the system with an excessively old virtual timestamp while preserving the effect of lag. The corresponding virtual deadline is then

$$VD_i = VE_i + \frac{\Delta \cdot S}{w_i}, \quad (7)$$

where S is an integer scaling constant used in place of fractional arithmetic. As in the analytical EEVDF rule, increasing w_i reduces the deadline increment, so higher-weight tasks return earlier in virtual order and obtain a proportionally larger share of processor time.

Task selection is realized through `sched_ext` dispatch queues. The scheduler maintains a single shared DSQ ordered by VD_i via `scx_bpf_dsq_insert_vtime()`, so dispatch from that queue implements earliest virtual deadline first. If `select_cpu()` finds an idle CPU, the task may instead be inserted directly into the local DSQ, reducing wakeup latency without changing the underlying policy.

Consumed service is charged in `stopping()`:

$$v_i \leftarrow v_i + \frac{\text{consumed} \cdot S}{w_i}. \quad (8)$$

This is the core fairness mechanism, so heavier tasks accumulate virtual time more slowly for the same physical runtime. Unlike the continuous model, `vtime_now` is not maintained by explicit integration of $1/W(t)$. It is advanced lazily in `running()` to at least the selected task’s virtual time.

4. Results. In this section, three benchmarks are selected for analysis: `hackbench` (1000 tasks), `sysbench` (default parameters) and `sched-latency`.

`Hackbench` is a well-known Linux scheduler benchmark designed to measure scheduler and inter-process communication (IPC) performance.

It creates groups of processes that exchange messages intensively, simulating high system load. The total execution time reflects how efficiently the scheduler handles context switching and task coordination.

Sysbench is a versatile benchmarking tool used to evaluate system performance across CPU, memory, and disk I/O. It generates computational workloads that simulate real-world usage scenarios, such as on-line transaction processing (OLTP) database operations, making it well suited for assessing overall system throughput under realistic conditions.

sched-latency is a self-written benchmark that uses eBPF tracing capabilities to measure scheduling latency. It attaches to kernel trace-points (`sched_wakeup`, `sched_switch`) and enqueue hooks to record the schedule delay for both the implemented `sched_ext` EEVDF scheduler and the baseline kernel EEVDF.

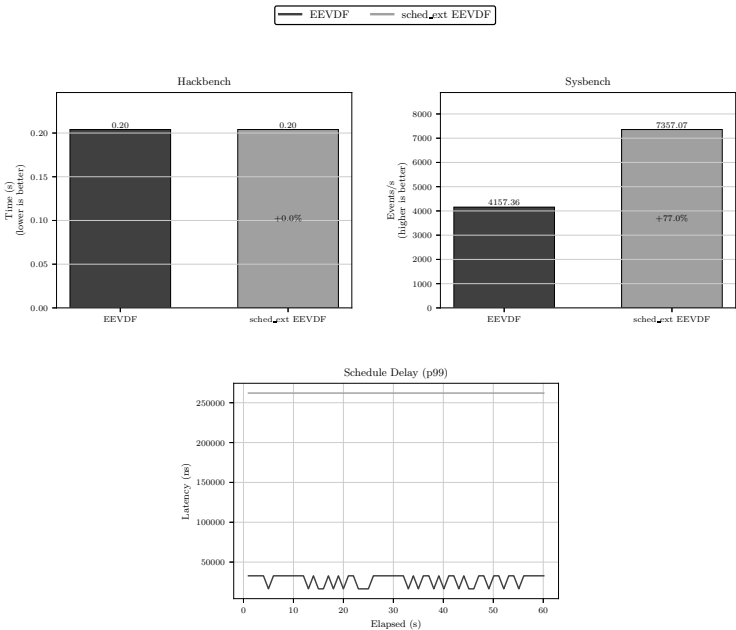


Figure 1. Test results for builtin Linux 7.0-rc4 EEVDF scheduler and implemented `sched_ext` EEVDF

5. Conclusion. The proposed sched_ext-based EEVDF implementation shows no measurable degradation in hackbench (0.20 s for both schedulers, +0.0%), demonstrating that the eBPF dispatch overhead is negligible for intensive, scheduler demanding workloads.

Sysbench throughput increased from 4157 events/s (default EEVDF) to 7357 events/s (sched_ext EEVDF), a +77% improvement, suggesting that the sched_ext variant may offer advantages for sustained computational workloads.

The 99th percentile scheduling latency of the sched_ext variant is approximately 250 μ s compared to 25 μ s for the in-kernel baseline, a 10 $\times$ increase attributable to the overhead of eBPF dispatch and contention on the single shared dispatch queue (DSQ).

However, the main achievement of this research and proposed sched_ext EEVDF is the flexibility of the resulting solution. Any constant or formula can be modified and tested without requiring a reboot, thanks to the sched_ext framework.

References

1. An EEVDF CPU scheduler for Linux [Internet resource]: <https://lwn.net/Articles/925371> (date 10.11.2025).
2. The extensible scheduler class [Internet resource]: <https://lwn.net/Articles/922405> (date 03.09.2025).
3. Humphries J. T., Natu N., Chaugule A., Weisse O., Rhoden B., Don J., Rizzo L., Rombakh O., Turner P., Kozyrakis C. ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling // Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21). 2021. P. 588-604.
4. Stoica I., Abdel-Wahab H. Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation. PhD thesis. USA: Old Dominion University, 1995. P. 1-37.
5. Stoica I., Abdel-Wahab H., Jeffay K., Baruah S., Gehrke J., Plaxton C. G. A Proportional Share Resource Allocation Algorithm for Real-Time, Time-Shared Systems // Proceedings of the 17th IEEE Real-Time Systems Symposium. 1996. P. 1-12.